bpmn.pic

`bpmn.pic` is a collection of DPIC macros for generating BPMN 2.0 diagram images for use in various kinds of documents and presentations.

Although the library focuses on supporting SVG as the primary output format, most of the elements (except pools and lanes) should also work with other formats produced by DPIC.

Currently supported BPMN element types include:

- **Events**: start, non-interrupting, catching, boundary, throwing, end;
- **Tasks**: none, receive, send, manual, service, user, script, rule;
- **Sub-Processes**: collapsed, sequential, parallel, loop, compensation, ad-hoc, expanded;
- **Gateways**: none, exclusive, parallel, inclusive, event-based (exclusive, parallel);
- **Flows**: sequence flow, default condition flow, conditional flow, message flow;
- **Pools**: horizontal pools and lanes, horizontal black box pools.

The goal of `bpmn.pic` is to provide just the visual layer of BPMN 2.0. If you need full support for BPMN 2.0-compatible XML, please try other tools (like bpmn.io) instead.

Resources

- DPIC documentation

Installation

Download `bpmn.pic` from here:

```
$ wget https://gitlab.com/pkierat/bpmn-pic/-/raw/master/bpmn.pic
```

The file is self-contained, and has no external run-time dependencies apart from DPIC itself.

Building from source

Buiding `bpmn.pic` requires the following tools to be available on the `PATH`:

- Binutils
- Make
- M4 Macro Processor

Procedure:

1. Clone the repository from GitLab or GitHub:

```
$ git clone git@gitlab.com:pkierat/bpmn-pic.git ./bpmn-pic
$ cd bpmn-pic
```

2. Build `bpmn.pic`:

```
$ make -C src bpmn.pic
```

3. (Optional) Build `README.pdf`:

```
$ make README.pdf
```

Usage

1. Include the `bpmn.pic` file in your PIC diagram using the `copy` statement:

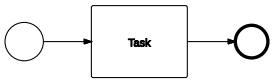
```
.PS
copy "bpmn.pic"

Start(none); SequenceFlow(); Task("Task"); SequenceFlow(); End(none)
.PE
```

2. Generate the SVG file:

```
$ dpic -v diagram.pic > diagram.svg
```

The output SVG should look like below:

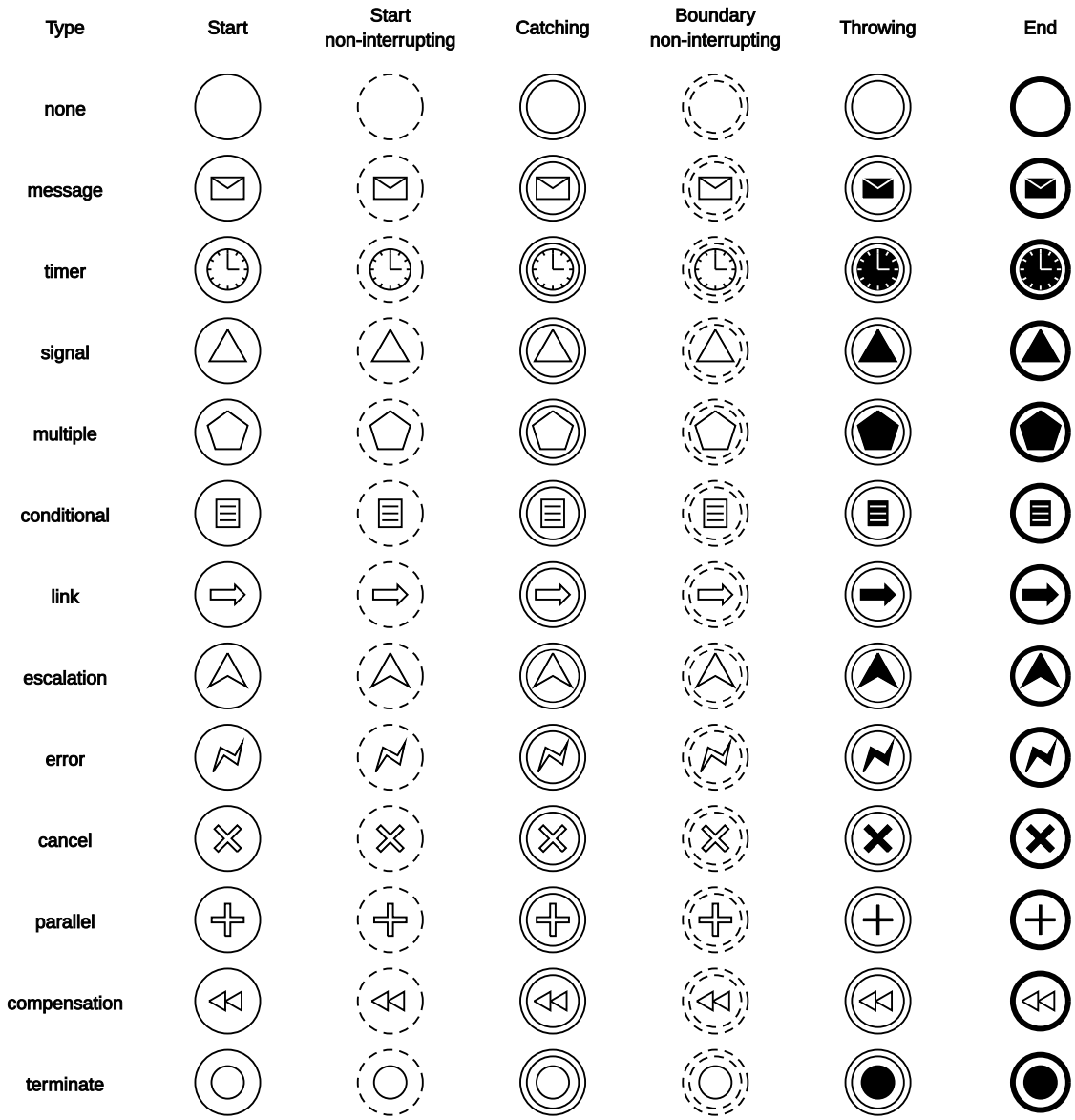


For more usage samples, please consult the `*.pic` files in the `img` directory.

Elements

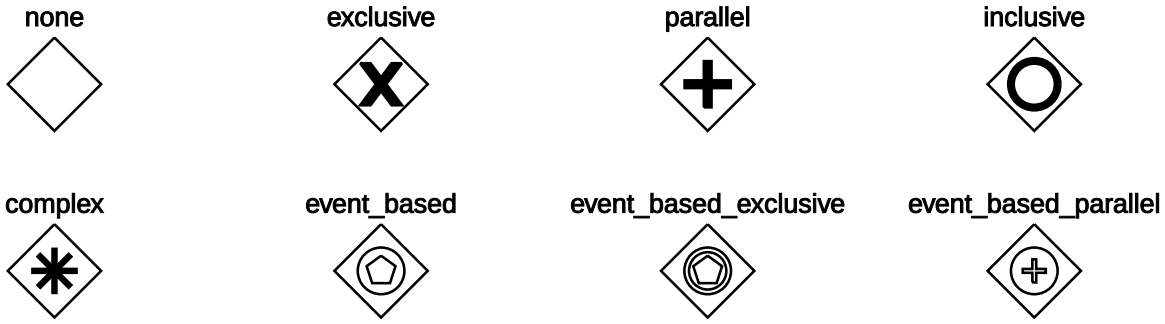
Events

```
Start(type [, attributes ])
StartNonInterrupting(type [, attributes])
Boundary(type [, attributes])
BoundaryNonInterrupting(type [, attributes])
Catching(type [, attributes])
Throwing(type [, attributes])
End(type [, attributes])
```



Gateways

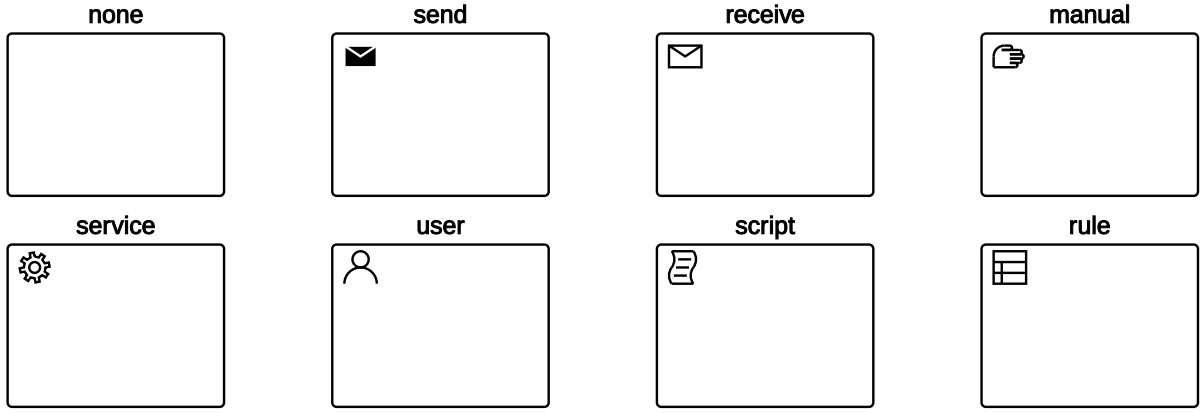
```
Gateway(type [, attributes ])
```



Activities

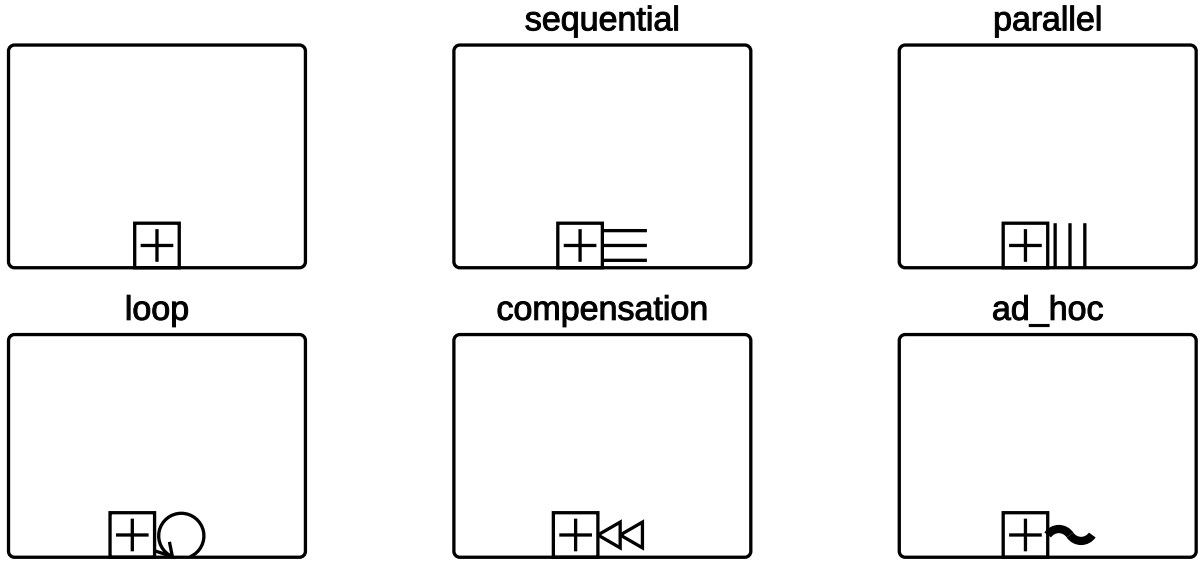
Tasks

```
Task(text [, type [, attributes ] ])
```



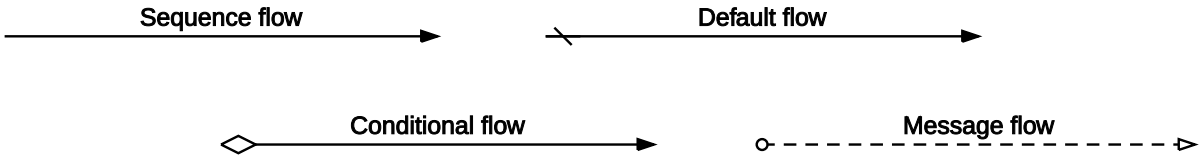
Sub-processes

```
SubProcess(text [, type [, attributes ] ])
```



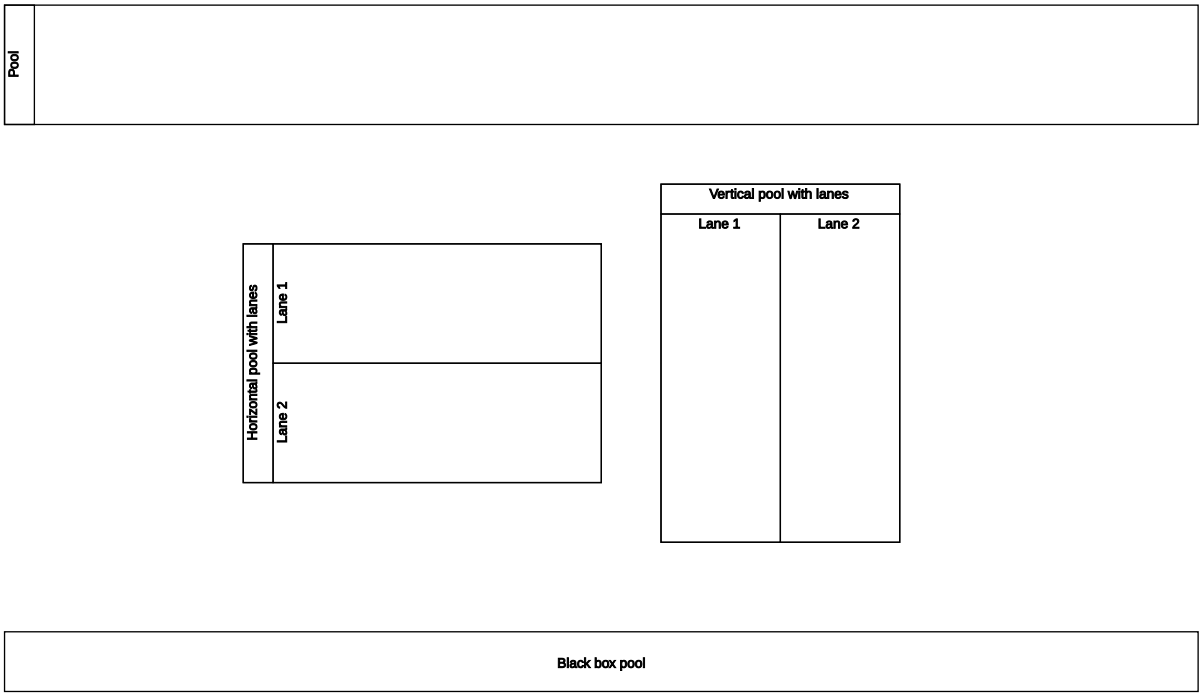
Flows

```
SequenceFlow([ linespec_1 [, linespec_2 [, ⋯ ] ] ])
DefaultFlow([ linespec_1 [, linespec_2 [, ⋯ ] ] ])
ConditionalFlow([ linespec_1 [, linespec_2 [, ⋯ ] ] ])
MessageFlow([ linespec_1 [, linespec_2 [, ⋯ ] ] ])
```



Pools and Lanes

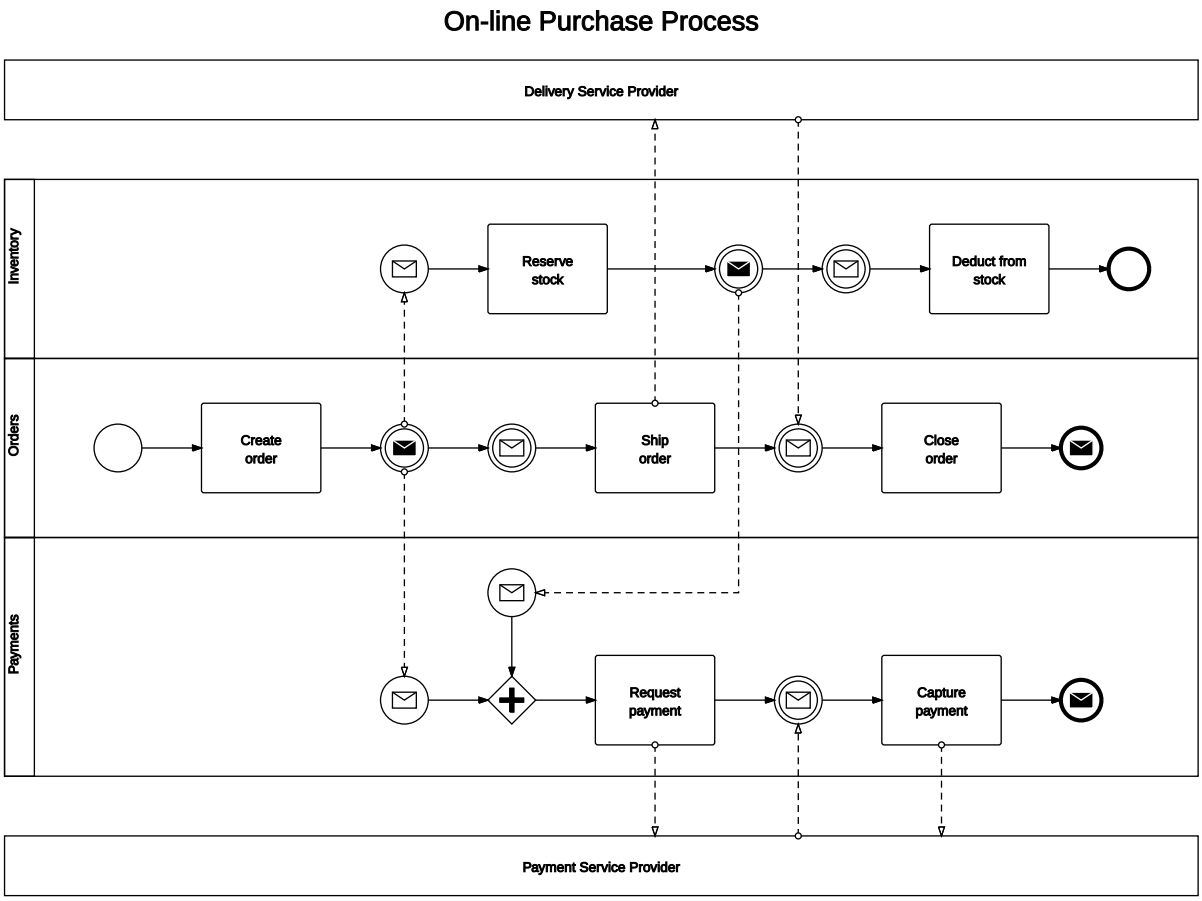
```
Pool(width, height, lanes [, attributes ])
BlackBoxPool(width, height [, attributes ])
Lane(height, process [, attributes ])
```



Text

```
Text(text, position [, rotation ])
```

Demo



Notes

1. `bpmn.pic` doesn't perform any kind of syntactic or semantic validation against the BPMN 2.0 spec. In particular, there's no check if a given combination of event type (start, end etc.) and trigger type (message, timer etc.) is valid according to the specification.
2. DPIC does not support text rotation, it needs to be implemented in SVG directly instead. For the rotated text to render properly, it must be created after its target location is determined, e.g.:

```
.PS
B: box width 0.25 height 1 ; { Text("Rotated text", B.c, "-90") }
.PE
```

   Also, for the same reason, the north-west corner of the entire diagram needs to be positioned at (0, 0). This can be achieved by positioning the first element with .nw at (0, 0) or by using the dedicated `Diagram(width [, title ])` macro:

```
.PS
Diagram(10, "Diagram")
P: Pool(10, 1, []) ; { Text("Pool", P.Label.c, -90) }
.PE
```